

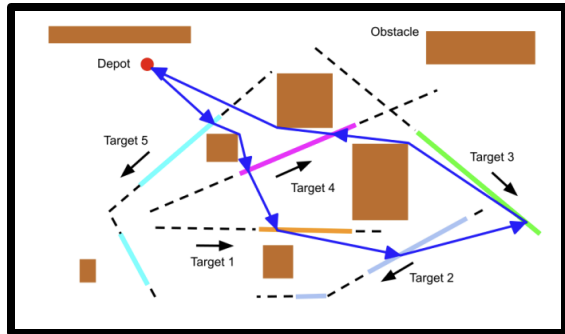
RMQ Final Presentation: Moving Target Visibility Graph

Robert LaBarge

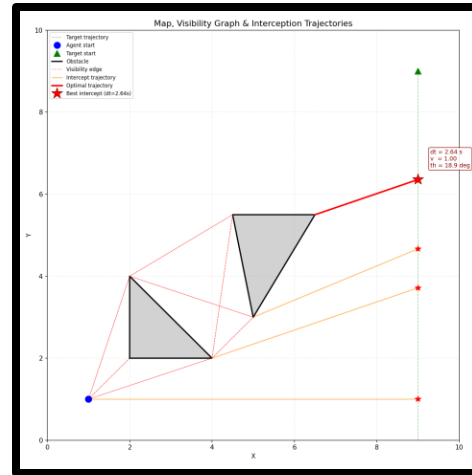
Quinten Pienaar

Michael Haines

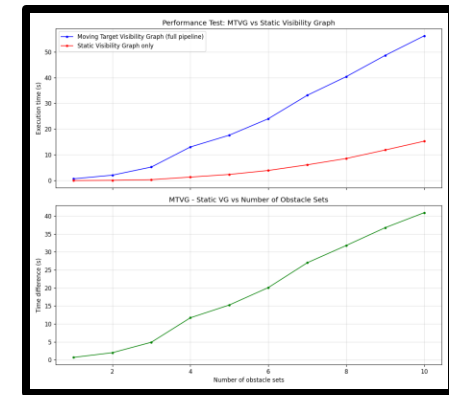
This presentation will cover a project overview, our methods, our results, and potential future steps



Project overview

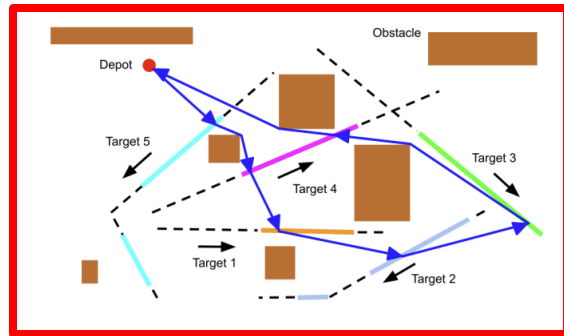


Method

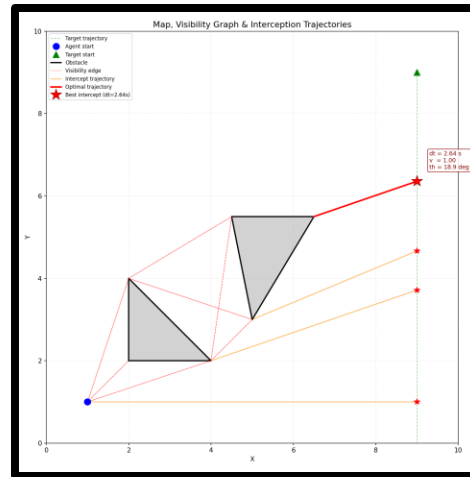


Results

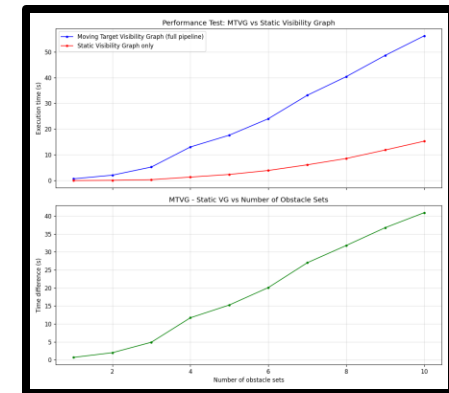
Let's begin with our project overview



Project overview



Method



Results

Motivation:

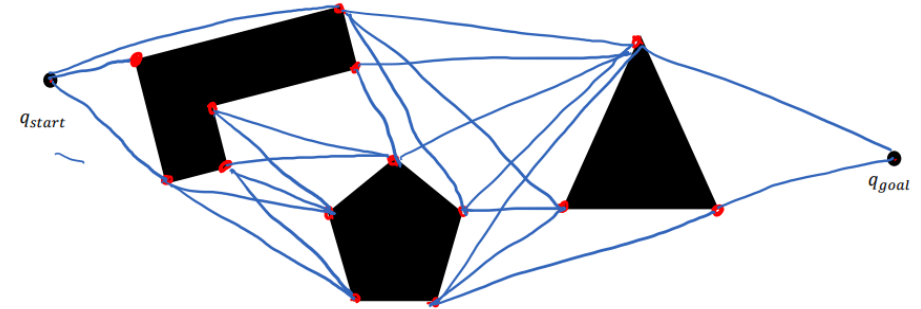
- Sometimes robots need to intercept targets that are moving
- Targets move, but they also exist in a space with obstacles
- Traditional path planning does not consider this movement, and cannot account for it



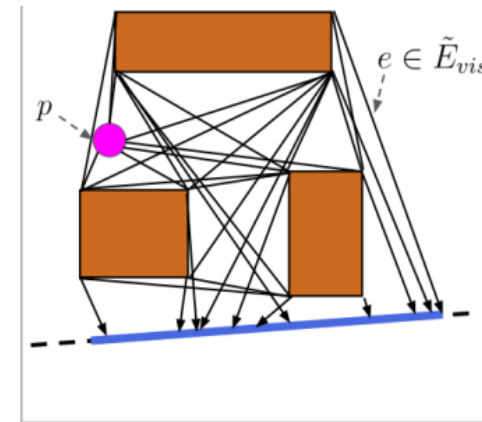
Example of robot with moving “goals”

Moving Target Visibility Graph

- Bhat et al. [1] proposed a complete solution to MT-TSP
- A key idea was a Moving Target Visibility Graph (MTVGG)
- This builds upon traditional visibility graph concept to consider the time-dependency of robot-to-goal interceptions



[2] Image of traditional visibility graph



[1] paper citation

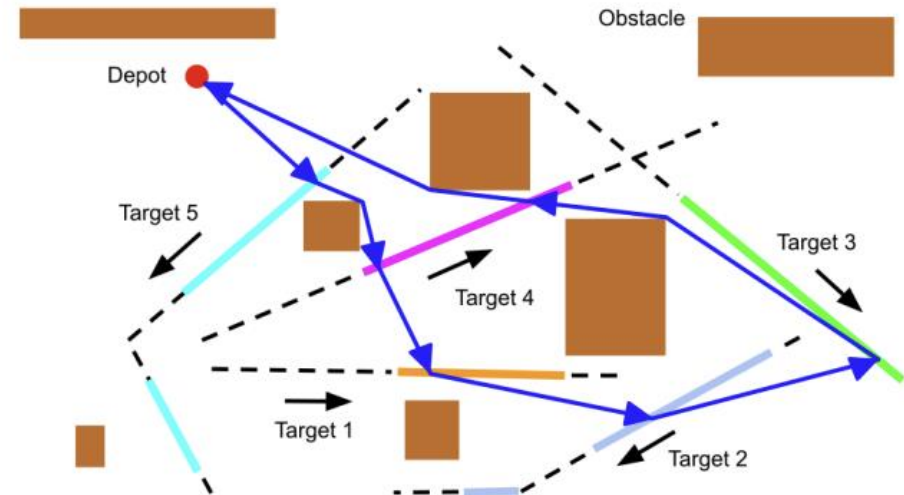
Scope

- [1] Implements MTVGs and path planning
- [1] Incorporates "interception windows"

- We only consider MTVGs – not path planning
- We assume constant, linear target and agent motion
- We included consideration for obstacles

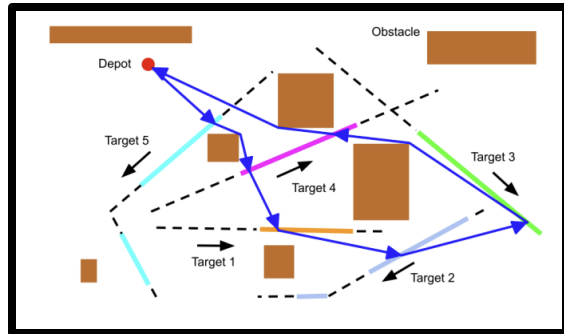
Goals

1. Implement a simplified MTVG
 - Considers linear motion and obstacles
2. Quantify performance
 - Feasibility
 - Computation time

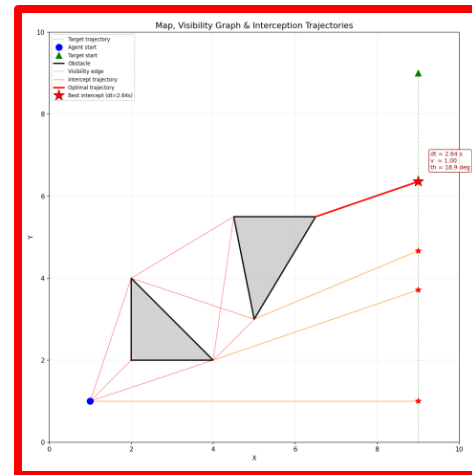


Robot moves through moving targets
[1]

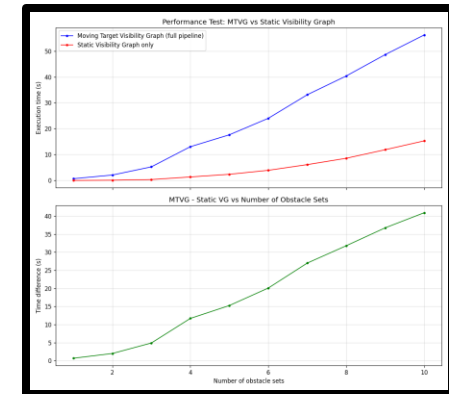
Now on to our methodology



Project overview



Method



Results

Method - object oriented programming to create MTVG

Problem consists of
two classes

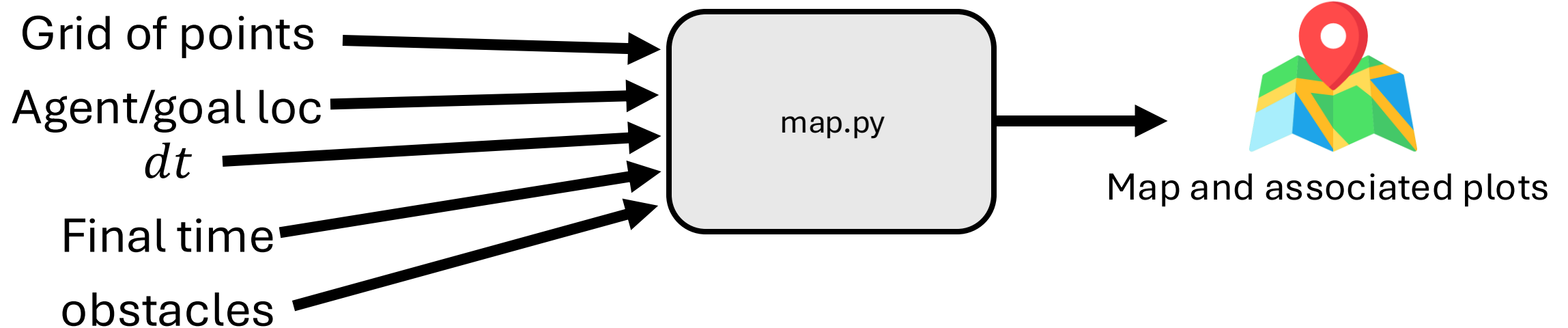


Agent



Map

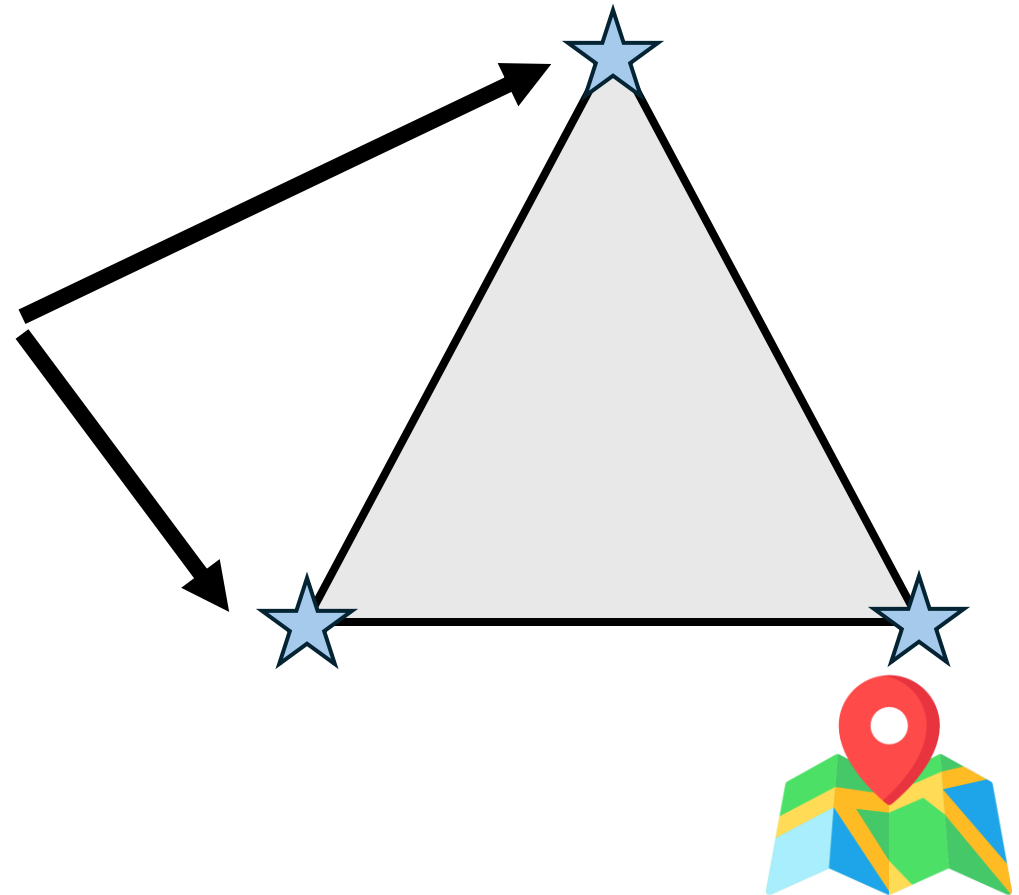
Map Class: 2d grid of points, target, agent



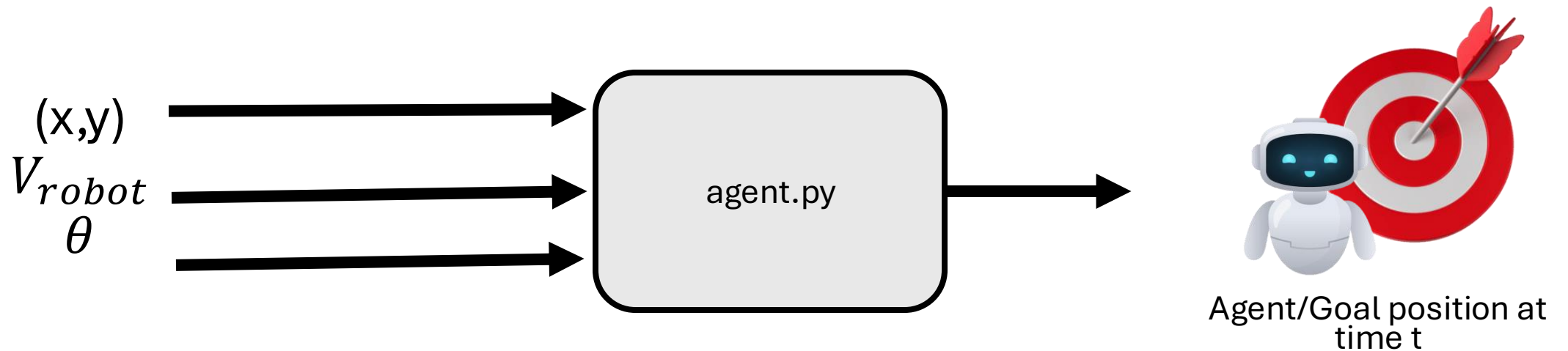
Obstacles are stored as an ordered 2d list of vertices in the map class

$$O = \{O1, O2, \dots, On\}$$
$$Oi = \{(x1, y1), \dots, (xm, ym)\}$$

This is then turned
into edges and nodes



Agent Class – An agent class defines both the target and agent



$$x_t = V \cos(\theta)t$$
$$y_t = V \sin(\theta)t$$

MTVG Algorithm

Algorithm 1 Construct Moving Target Visibility Graph (MTVG)

Require: Start position p , target trajectory $\tau(t)$, obstacle set $\mathcal{O}$, max speed

$v_{\max}$

Ensure: Moving Target Visibility Graph $\tilde{G} = (\tilde{V}, \tilde{E})$

- 1: Extract all obstacle vertices V_O
- 2: $V_{\text{vis}} \leftarrow V_O \cup \{p\}$
- 3: Initialize $E_{\text{vis}} \leftarrow \emptyset$
- 4: **for** each pair $(u, w) \in V_{\text{vis}} \times V_{\text{vis}}$ **do**
- 5: **if** line segment $[u, w]$ lies entirely in free space **then**
- 6: Add edge (u, w) to E_{vis}
- 7: **end if**
- 8: **end for**
- 9: Define moving target node s
- 10: $\tilde{V} \leftarrow V_{\text{vis}} \cup \{s\}$
- 11: $\tilde{E} \leftarrow E_{\text{vis}}$
- 12: **for** each $q \in V_{\text{vis}}$ **do**
- 13: Compute visibility interval:

$$I_q = \{t \geq 0 \mid \text{line segment } [q, \tau(t)] \text{ is collision-free}\}$$

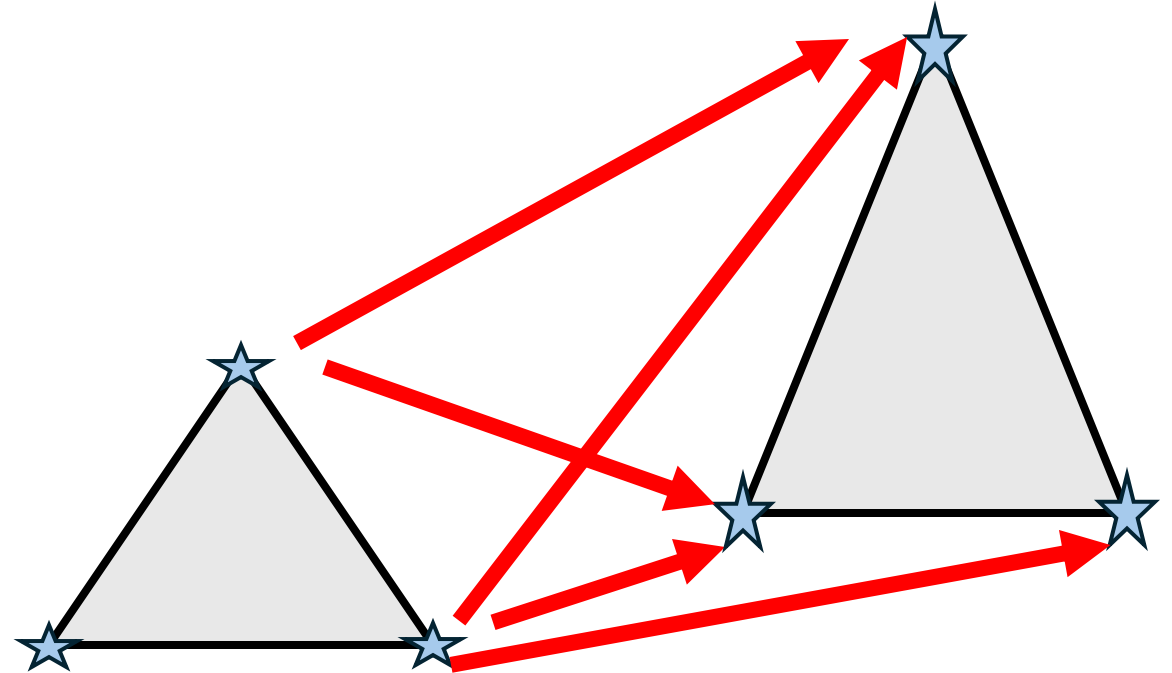
- 14: **if** $I_q \neq \emptyset$ **then**
- 15: Compute edge cost $c(q, s, t)$:

$$c(q, s, t) = \min_{\Delta t \geq 0} \{\Delta t \mid t + \Delta t \in I_q, \|\tau(t + \Delta t) - q\| \leq v_{p \max} \cdot \Delta t\}$$

- 16: Add structured edge $(q, s, I_q, c(q, s, t))$ to $\tilde{E}$
 - 17: **end if**
 - 18: **end for**
 - 19: **return** $\tilde{G} = (\tilde{V}, \tilde{E})$
-

Visibility graph

- Stored as a set in the map class
- An edge is valid if a straight line can be drawn between vertices without a collision with an obstacle
- $O(Nn^2)$ time complexity



```
def build_visibility_graph(self):  
    print("Building visibility map")  
    # for point in obstacles and agents  
    for pt1 in self.Vis:  
        # for another point in obstacles and agents  
        connected_nodes = self._find_connected_nodes(pt1)  
  
        for node in connected_nodes:  
            line = (pt1, node)  
            self.Edges.add(line)
```

Visibility intervals

- Dictionary in map class:
 - Key – Nodes on the map
 - Value – list of time intervals where node can "see" the target
- $O(N_n * N_t * \text{Time_Steps})$ time complexity

```
def build_visibility_intervals(self):  
    print("Creating visibility intervals")  
    # plot target trajectory  
    t = 0  
    visibility_intervals = {}  
    previous_nodes = set()  
  
    while t < self.tf:  
        t += self.dt  
        pt1 = self.target.find_position(t)  
        # find connected nodes  
        connected_nodes = self._find_connected_nodes(pt1)
```

Edge Cost

- Dictionary in map class:
 - Key – node
 - Value – edge cost

For each node a cost is computed:

$$z = [\text{interception time } \Delta t, v_a, \theta_a]$$
$$t \in [t_o, t_f], 0 \leq v_a \leq v_{max}, \theta_a \in [0, 2\pi]$$

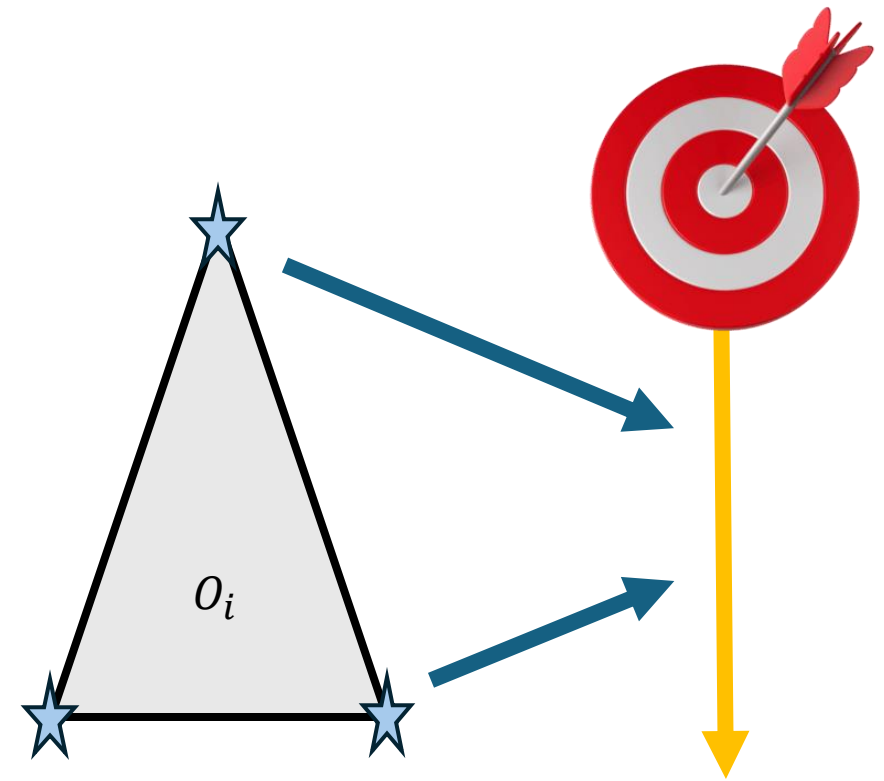
The solver than minimizes interception time:

$$\min(\Delta t)$$

z

Such that the agent and target position coincides at time $t + \Delta t$

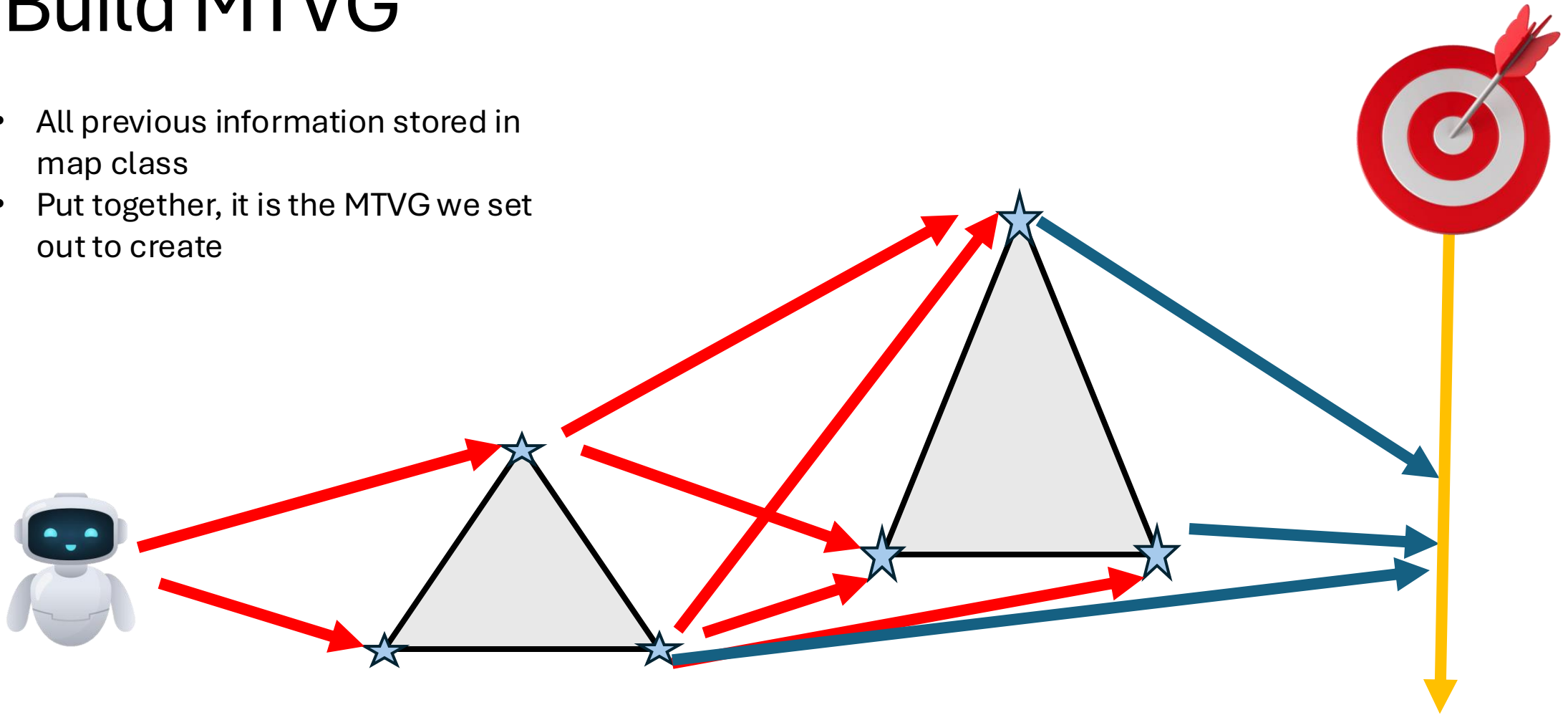
$O(Nn \cdot Nt \cdot \text{Time_Steps}/2)$ time complexity



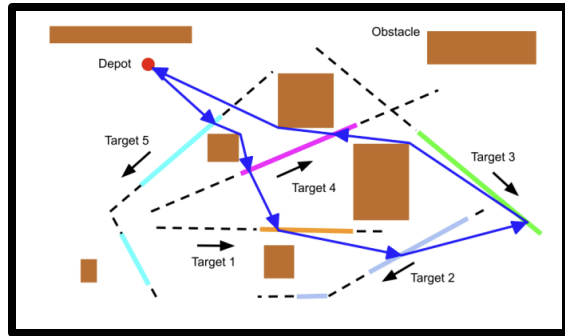
```
def compute_edge_cost(self, visibility_intervals):  
    ...  
    Compute the minimum cost to intercept target from each node  
  
    Inputs:  
    | visibility_intervals: dictionary - each nodes visibility time intervals  
  
    Outputs:  
    | edge_costs: dictionary - each nodes edge cost and interception location  
    | ...  
  
    print("Computing edge costs")  
    edge_costs = {}  
    # loop over each node  
    for node, interval in visibility_intervals.items():  
        edge_costs[node] = (1e10, 0, 0) # initialize to large edge cost  
        # for each visibility interval find closest interception point  
        for dt in interval: # loop over each time interval  
  
            # calculate time to intercept  
            result = self._find_intercept(node, dt)
```


Build MTVG

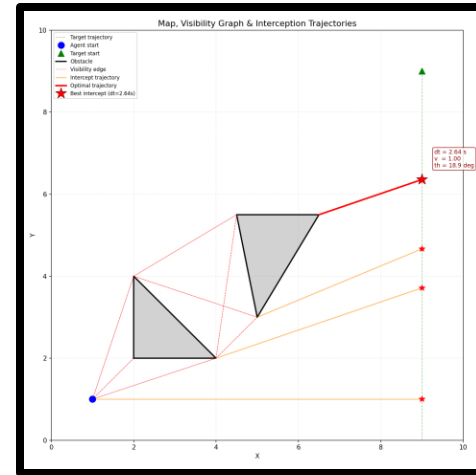
- All previous information stored in map class
- Put together, it is the MTVG we set out to create



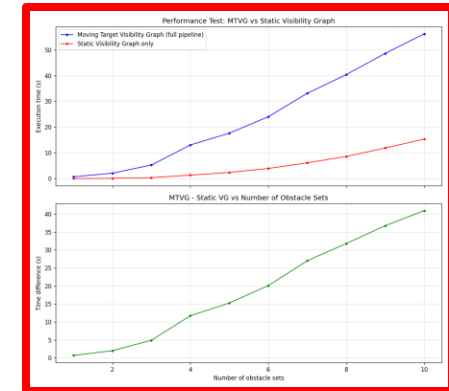
Let's move on and discuss our results



Project overview

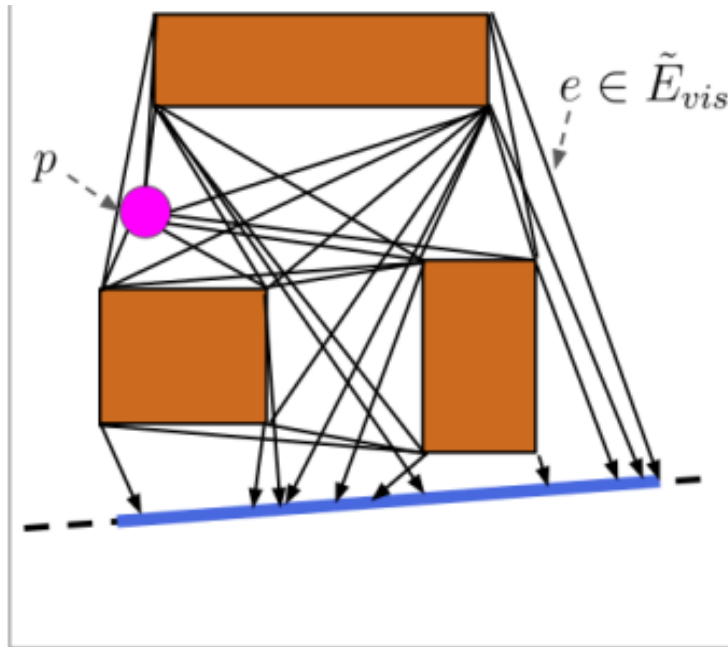


Method

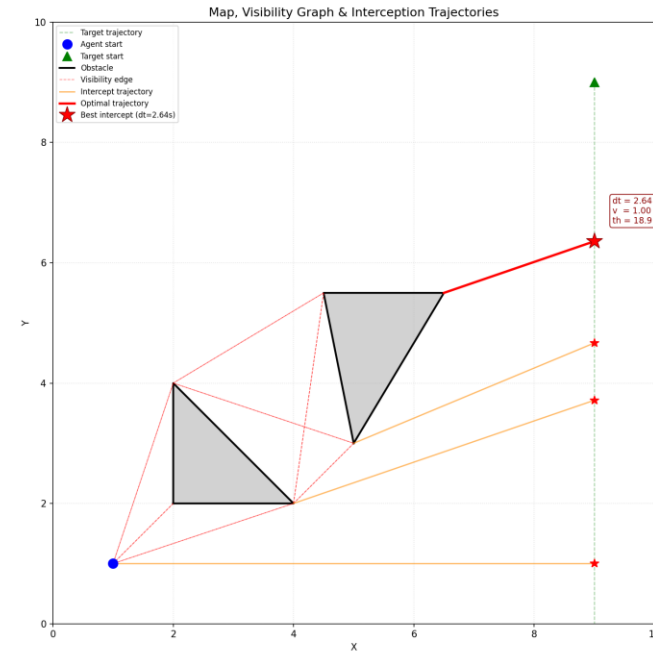


Results

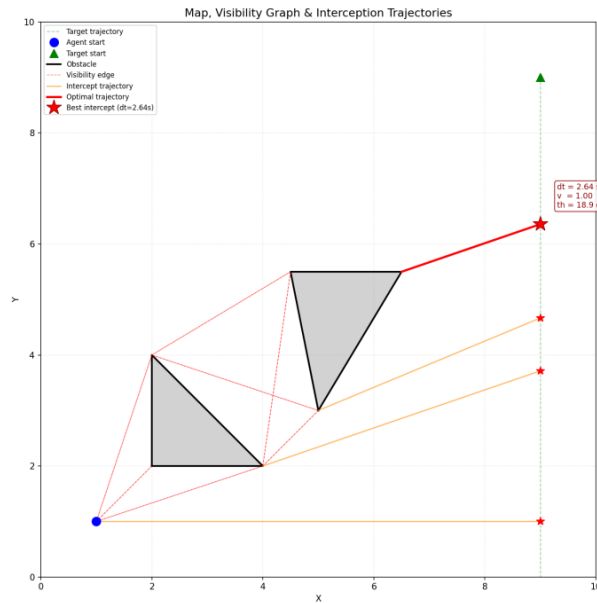
Results – The paths generated are consistent with the model



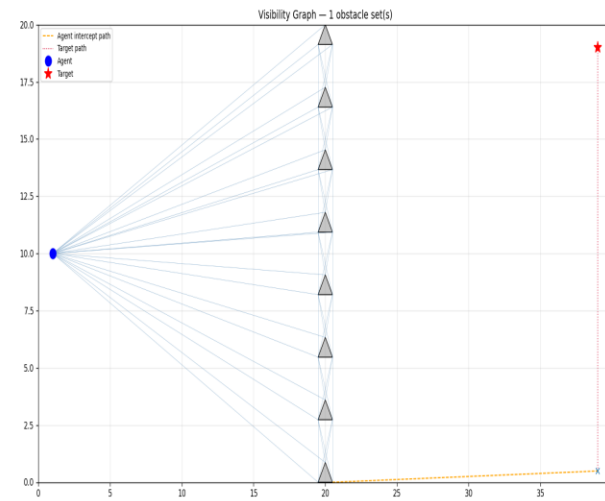
[1]



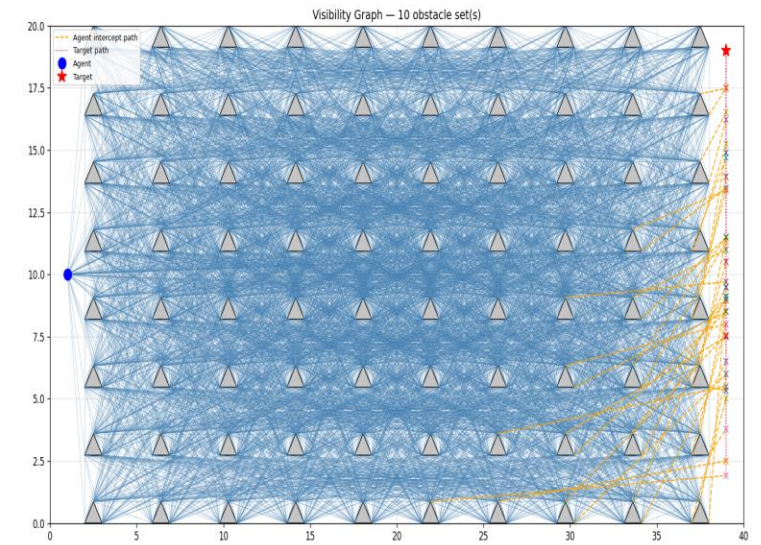
Results – MTVG is more expensive than the static VG



2 obstacles



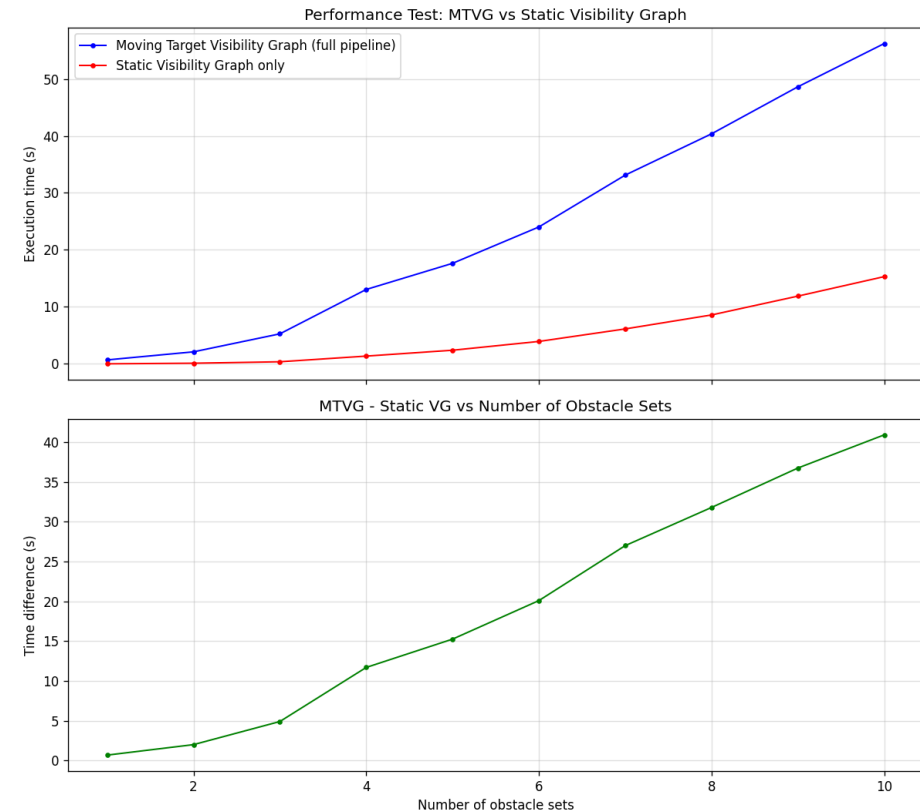
8 obstacles



80 obstacles

Results – Compared to the method in class, this is far more expensive

- Overall time complexity:
 $O(Nn^2 + (3/2)*Nt*Nn*Time_Steps)$
 - Construction of baseline visibility graphs still dominates time complexity
 - Number of nodes in map next largest contributor
-
- Our MTVG is "embarrassingly parallel"
 - All computations are cheap
 - If scaled, would likely be memory bound
 - Each operation requires full knowledge of world map

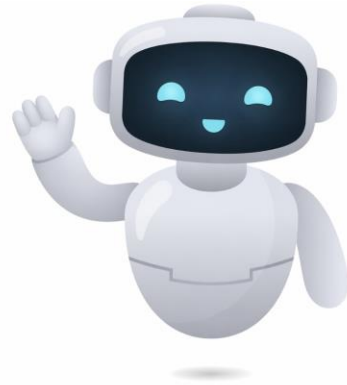


Future Steps

- Parallelize
- Test with complex obstacles and target trajectories
- Implement path planning
- Real-world implementation

References:

- [1] A. Bhat, G. Gutow, B. Vundurthy, Z. Ren, S. Rathinam, and H. Choset, "A Complete Algorithm for a Moving Target Traveling Salesman Problem with Obstacles," arXiv:2409.09852v3, October 2024.
- [2] E. Yel, "Robotics II Lecture Slides," Rensselaer Polytechnic Institute, Robotics II, Spring 2026.



Thank you!

We would now like to open the floor to any questions

